

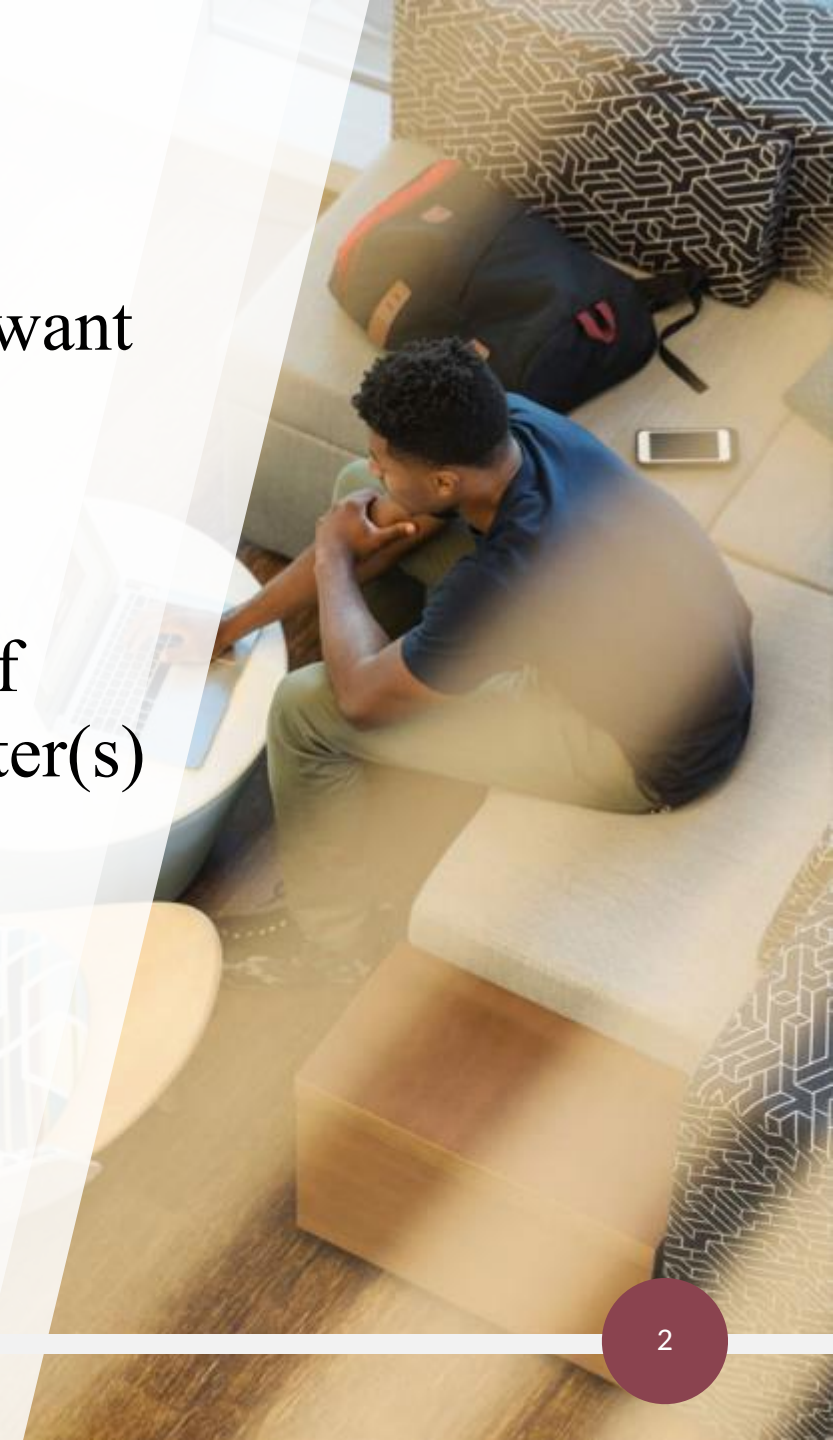
A photograph of four students in a library setting. A young woman on the left is smiling and looking towards the center. Next to her is a young man with dark hair, also smiling and looking towards the right. On the right side, a young woman with glasses is looking at a laptop screen. In the foreground on the right, the back of a young man's head and shoulders are visible as he looks towards the laptop. They are all gathered around a table with a laptop, books, and papers. The background is filled with bookshelves. A semi-transparent blue diagonal band runs across the image, and a semi-transparent red horizontal band is at the bottom.

Java 25

Scoped Values

Scoped Values

- Assume for a moment that you have some contextual information, such as a user ID or a request ID, that you want to share across many method calls in your call stack.
- One option is to pass this information to each method requiring it, but this requires modifying the signatures of each of the affected methods to include the new parameter(s) - which is not very elegant and clutters your code.



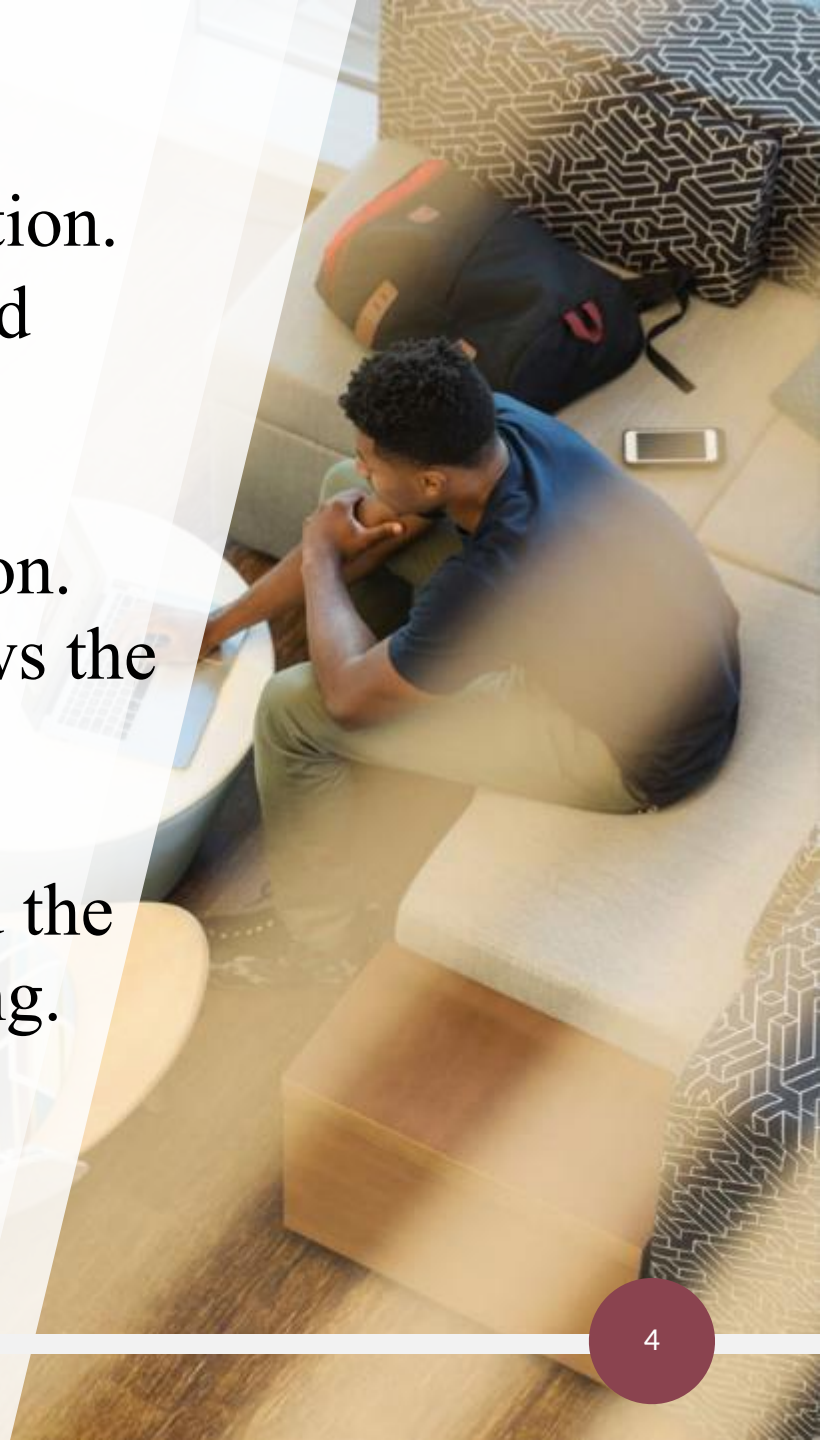
Scoped Values

- Traditionally, `ThreadLocal` was used to attach context to a thread and read it later.
- However, `ThreadLocal` has issues:
 - mutable - the binding could be accidentally overwritten.
 - leakage - could forget to tidy up when finished; sensitive data could leak beyond the scope intended.
 - `ThreadLocal` does not fit well with the newer concurrency models, especially virtual threads.



Scoped Values

- Java 25 adds `ScopedValue` for just this type of situation. Consider it like a map, where the scoped value is mapped (“bound”) to a context value.
- This binding applies for a *scope*, i.e. a block of execution. The scope is dynamic and not lexical in nature - it follows the call-stack and not the curly braces.
- The binding is read-only within that scope: we can read the context value, we can’t accidentally overwrite the binding.
- When the code block finishes, the scoped value is automatically unbound (addressing leakage concerns).



Scoped Values

- Scoped values can be rebound in a nested scope - the inner binding temporarily overrides the outer one, and the outer binding becomes visible again when the inner scope ends.
- Note, even though the binding is read-only within a scope (nested scopes can rebound), the objects you bind to should ideally be immutable. For example, bind to a `String` rather than a `StringBuilder`.

